FPGA-BASED REAL-TIME OSCILLOSCOPE

Tanvir Kaur
&
Ethan Spooner

April 18, 2026

—

DIGITAL SYSTEM DESIGN

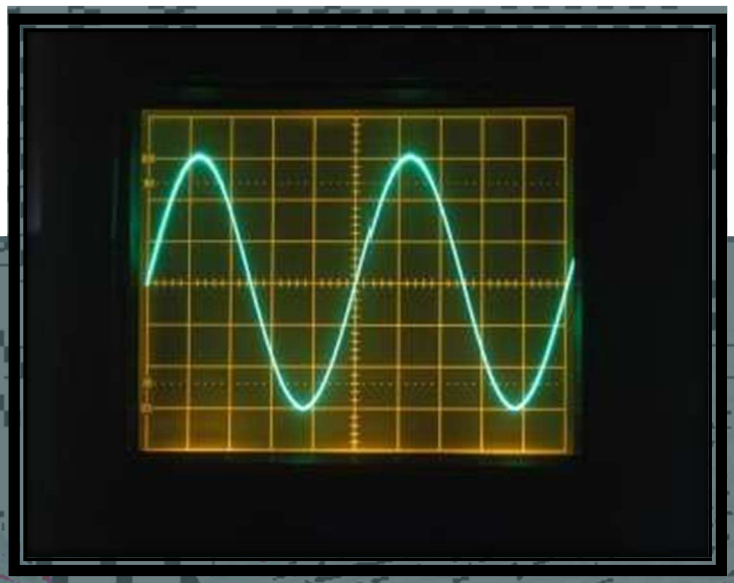
—

Professor Eduardo Casas

INTRODUCTION

We did this project because we wanted to deepen your understanding of digital systems.

Oscilloscopes combine ADCs, buffering, triggering, and display logic. It is one of the most fundamental tools in electronics, and building one from scratch provides a deeper understanding of how measurement instruments work internally. Instead of treating it as a black box, this project helped us reverse-engineer its essential functions and re-implement them using the DEO-Nano-SoC FPGA.



Objective

The objective of this project was to design, implement, and demonstrate a functional real-time digital oscilloscope using the FPGA fabric of the DE0-Nano-SoC. Our goal was to build a complete hardware-based signal acquisition and visualization pipeline capable of sampling an analog input, buffering the data, detecting trigger events, and rendering the waveform on a VGA display without relying on software intervention.

In the final implementation, the system successfully acquired analog samples from the TI Educational BoosterPack's LTC2308 ADC, processed them in real time, and displayed the waveform using a custom RTL VGA controller. The oscilloscope supported essential measurement features, including adjustable trigger level, selectable trigger modes, a configurable timebase for horizontal zoom, and vertical scaling for amplitude adjustment. All core signal-processing components such as sampling, timebase control, buffering, triggering, and waveform rendering were implemented entirely in SystemVerilog.

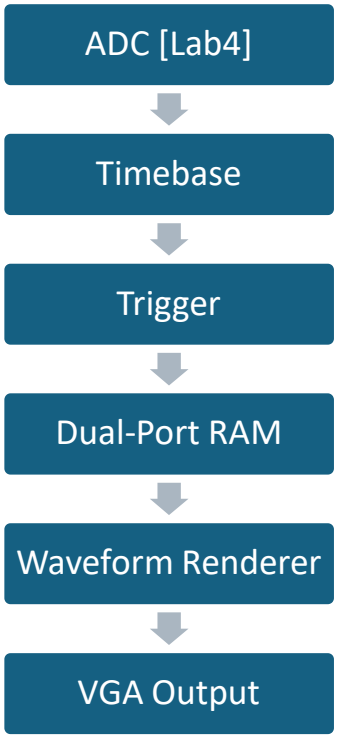
By the end of the project, we demonstrated a working hardware pipeline from analog input to digital display. Although not every planned feature behaved perfectly in all configurations, the system successfully showcased real-time acquisition, trigger-aligned waveform display, and user-controlled visualization parameters, meeting the primary learning and functional objectives of the project.

Detailed Description (Processes)

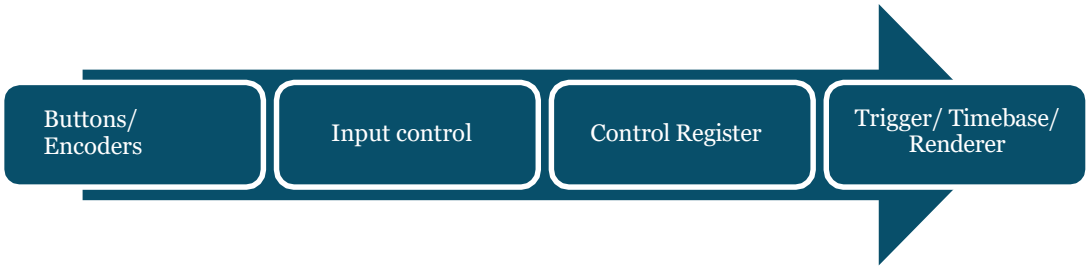
Oscilloscope Architecture

These two paths operate in parallel, allowing the oscilloscope to update the display in real-time while responding instantly to user input.

Data Path



Control Path



Control Register Map

Register	Width	Purpose
TRIG_LEVEL	12 bits (same as ADC bus)	Sets the ADC threshold
TRIG_MODE	2 bits	Select trigger type: Rising, Falling, Level trigger
TIMEBASE_SEL	8 bits	Controls sampling divider for horizontal zoom
SCALE_LEVEL	2 bits	Vertical scaling (waveform height adjustment)
CONTROL	2 bits	Reset, manual trigger, mode cycling

Implementation Details

ADC Sampling

We used the LTC2308 ADC interface from Lab 4. The ADC outputs a 12-bit sample and a valid signal. Samples are written into RAM whenever **sample_tick** is asserted by the timebase module.

The ADC runs continuously at 50 MHz, and the oscilloscope samples at a user-controlled rate determined by the timebase divider.

Timebase (Horizontal Zoom)

The timebase module divides the 50 MHz clock to control how often a new sample is written into RAM. The oscilloscope always displays **256 samples**, but the time between samples changes.

- Small divider: fast sampling ~ zoomed in
- Large divider: slow sampling ~ zoomed out

This allows the user to control how much time the 256 samples represent.

Trigger System

The trigger module supports four modes:

- **Rising edge:** *prev* < *level* AND *current* ≥ *level*
- **Falling edge:** *prev* > *level* AND *current* ≤ *level*
- **Level:** *current* == *level*
- **Always:** free-run mode

When a trigger event occurs, the write pointer resets to zero. This aligns the waveform so that the trigger point appears at the left edge of the display window, producing a stable waveform.

A one-clock pulse ensures the trigger fires only once per crossing.

Dual-Port RAM Buffer

We implemented the circular buffer using Altera dual-port RAM IP:

- **Write port (50 MHz):** Writes samples at each **sample tick**.
- **Read port (25 MHz):** Reads samples for VGA rendering.

The write pointer wraps around naturally due to the RAM depth. The read address is computed per pixel in the renderer.

This design allows safe crossing between the sampling and VGA clock domains.

Waveform Renderer

The waveform renderer is responsible for converting stored samples into VGA pixel data.

Key functions:

- Captures the newest sample pointer (*write_addr_sync*) at the start of each VGA frame.
- Computes the read address for each x-pixel: *read_addr = write_addr_sync - (X_RIGHT - xpixel)*
- Applies vertical scaling using *scale_level*.
- Draws:
    - waveform (green)
    - trigger line (color-coded)
    - oscilloscope border (white)
    -

This module produces the final RGB output for the VGA controller.

VGA Controller

We implemented a full 640×480 @ 60 Hz VGA timing generator:

- 25 MHz pixel clock
- Horizontal: 800 cycles
- Vertical: 525 lines
- Visible region: 640×480

The controller outputs RGB, HSYNC, and VSYNC signals to the monitor. The oscilloscope window is drawn inside this visible region.

User Input System

- KEY[1] — Vertical Scale

Each press increments the vertical scale level [1-3]. The renderer multiplies the ADC sample by this factor before mapping it to a y-pixel.

- s1 — Reset

Active-low reset that clears all registers and pointers.

- s2 — Manual Trigger

Forces a trigger event regardless of signal conditions.

- Rotary Encoder #1 — Trigger Level

Adjusts the trigger threshold.

- Rotary Encoder #2 — Timebase

Adjusts the sampling divider for horizontal zoom.

- Trigger Mode Button

Cycles through rising, falling, level, and always modes.

Results

This section of the report will show some final results of the project and will explain the images.

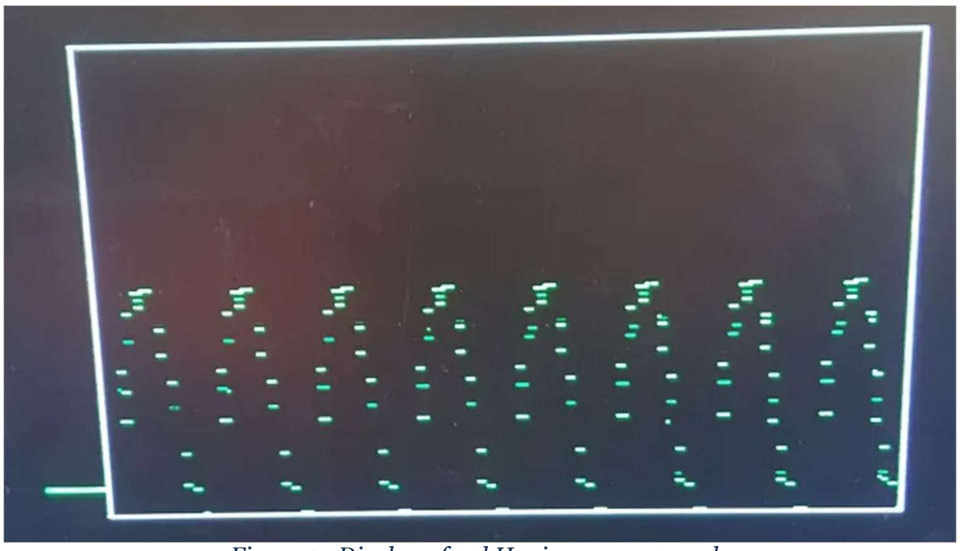


Figure 1 - Display of 30kHz sine wave, 1x scale

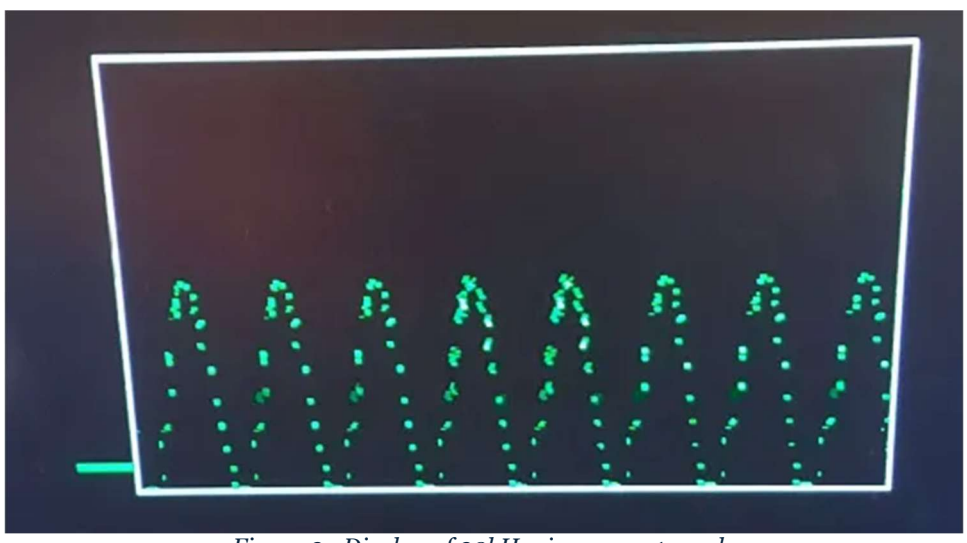


Figure 2 - Display of 20kHz sine wave, 1x scale

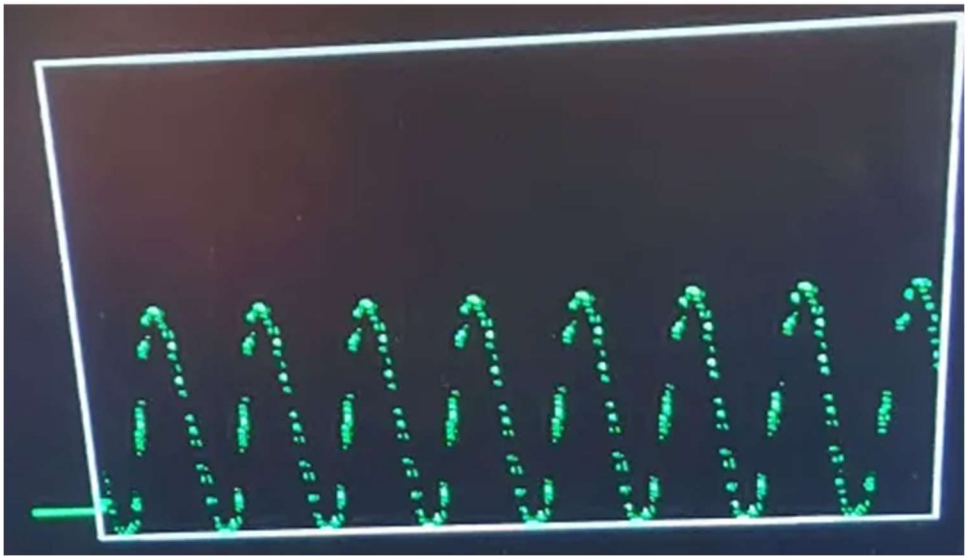


Figure 3 - Display of 10kHz sine wave, 1x scale

In all the display images you will notice some things, the display line is choppy and a line to the left outside of the screen. The line outside of the display screen is the trigger level indicator, it's a visual indicator of where the trigger level is being indicated. The reason the display line is choppy is because we only show the data at that instance. This creates a choppy effect as pixels in between the samples are not drawn in.

All the project milestones were completed; there is both time scaling and vertical scaling. Waveforms can be seen with no buffering on the display. Along with variable trigger level. This was all demonstrated during the project presentation. Additionally, during the presentation different types of signals such as various types of sawtooth and pulse signals were also detected with this code. Images of these signals are not shown as we forgot to take pictures of them.

Code

This section will show the code for the project. Code will be organized by the system Verilog modules. For easier reading experience.

oscilloscope_top.sv

```
// oscilloscope_top.sv

module oscilliscope_top (
		input logic clk50, s1, s2, // clk, Reset, Trigger
		//Encoders
		input  logic enc1a, enc1b,// Encoder #1 (trigger level)
		input  logic enc2a, enc2b, // Encoder #2 (timebase)
		//VGA
		output logic [13:0] GPIO_0,
		input logic [1:0] KEY,
		// ADC
		input  logic ADC_SDO,
		output logic ADC_CONVST, ADC_SDI, ADC_SCK
	) ;

	localparam SYS_CLK = 50_000_000;

	localparam DATA_WIDTH = 12;
	localparam ADDR_WIDTH = 8;
```

```
// Logic Values  ---------------------------------------------

// Buttons
logic reset, trigger;

// ADC
logic [DATA_WIDTH-1:0] adc_data;

// Input_Control
logic [DATA_WIDTH-1:0] trig_level;
logic [1:0] trig_mode;
logic [DATA_WIDTH-1:0] timebase_sel; // Can change
logic [1:0] scale_level;

// Trigger
logic trigger_fire;

// Timebase Module //Can Change
logic sample_tick;

// Circular Buffer
logic [DATA_WIDTH-1:0] buffer_data;
logic [ADDR_WIDTH-1:0] read_addr;

// Waveform Renderer
logic [2:0] rgb;
logic [9:0] xpixel, ypixel;
logic rendering;


// CLOCKS & COUNTERS ---------------------------------------------

// Encoder Clock -> 1kHz
logic clk1;
clkdiv
c_1k(
	.clkin(clk50),
	.clkout(clk1)
);

// VGA Clock
logic clkvga;
clkdiv #(.fin(SYS_CLK), .fout(25_000_000))
c_vga(
	.clkin(clk50),
	.clkout(clkvga)
);

// Write address counter
logic [ADDR_WIDTH-1:0] write_addr, write_addr_sync;
always_ff @(posedge clk50) begin
	if (reset)
		write_addr <= 0;

	else if (trigger_fire)
		write_addr <= 0;
```

```
		else if (sample_tick)
			write_addr <= write_addr + 1;
	end

	always_ff @(posedge clkvga)
		write_addr_sync <= write_addr;

	// INPUTS ---------------------------------------------

	// Button Inversions
	assign reset = !s1;
	assign trigger = !s2;

	// ADC
	ltc2308
	ltc2308_0 (
		.clk(clk50),
.reset(reset),
.sdo(ADC_SDO),
.convst(ADC_CONVST),
.sdi(ADC_SDI),
.sck(ADC_SCK),
		.sample_out(adc_data),
		.adc_ready()
	);

	// Rotary Logic
	input_control #(.DATA_WIDTH(DATA_WIDTH), .ADDR_WIDTH(ADDR_WIDTH))
	input_0(
		.clk(clk1),
		.reset(reset),
		.trigger(trigger),
		.enc1a(enc1a), .enc1b(enc1b),// Encoder #1 (trigger level)
		.enc2a(enc2a),.enc2b(enc2b), // Encoder #2 (timebase)
		.scale(KEY[1]),
		.trig_level(trig_level),
		.trig_mode(trig_mode),
		.timebase_sel(timebase_sel),
		.scale_level(scale_level)
	);

	// Computation ----------------------------------------

	// Trigger
	trigger #(.DATA_WIDTH(DATA_WIDTH), .ADDR_WIDTH(ADDR_WIDTH))
	trig_0(
		.clk(clk50),
		.reset(reset),
		.data_in(adc_data),
		.trig_level(trig_level),
		.trig_mode(trig_mode),
		.trigger_edge(trigger_fire)
	);

	// Timebase module
	timebase
	time_0(
```

```
		.clk(clk50),
		.reset(reset),
		.timebase_sel(timebase_sel),
		.sample_tick(sample_tick)
	);


	// Stored values ----------------------------------------------------
	// if all else fails
//	circularBuffer #(.DATA_WIDTH(DATA_WIDTH), .ADDR_WIDTH(ADDR_WIDTH))
//	cb_0 (
//		.clkin(clk50),
//		.clkout(clkvga),
//		.reset(reset),
//		//Write Logic
//		.write_en(sample_tick),
//		.write_addr(write_addr),
//		.data_in(adc_data),
//
//		//Read Logic
//		.read_en(rendering),
//		.read_addr(read_addr),
//		.data_out(buffer_data)
//	);

	// Altera Config -> Dual Port RAM
	dual_port_ram
	dp_0 (
		.data(adc_data),
		.rdaddress(read_addr),
		.rdclock(clkvga),
		.wraddress(write_addr_sync),
		.wrclock(clk50),
		.wren(sample_tick),
		.q(buffer_data)

	);

	// Display ----------------------------------------------------

	// Renders the Waveform
	waveform_renderer
	wve0(
		// Essential
		.clk(clkvga),
		.xpixel(xpixel),
		.ypixel(ypixel),
		.data_in(buffer_data),
		.write_addr(write_addr_sync),
		.outputRGB(rgb),
		.read_addr(read_addr),
		// Non Essential
		.trig_level(trig_level),
		.trig_mode(trig_mode),
		.time_scale(timebase_sel),
		.scale_level(scale_level)
	);
```

```
	// Controls VGA Port - does the actual displaying
	vgacontroller
	vga0(
		.clk(clkvga),
		.reset(reset),
		.inputRGB(rgb),
		.vsync(vsync), // Hide?
		.hsync(hsync), // Hide?
		.xpixel(xpixel),
		.ypixel(ypixel),
		.gpio(GPIO_0)
	);

endmodule
```

waveform_renderer.sv

```
// waveform_renderer.sv
// By: Ethan Siemens

module waveform_renderer
#( DATA_WIDTH = 12, ADDR_WIDTH = 8 )
(  // Essentials
	input logic clk, // VGA Clock
  input logic [9:0] xpixel, // Where is X pixel
	input logic [9:0] ypixel, // Where is y pixel
	input logic [DATA_WIDTH-1:0] data_in,
	input logic [ADDR_WIDTH-1:0] write_addr,
	output logic [2:0] outputRGB,
	output logic [ADDR_WIDTH-1:0] read_addr,

	// Extras
	input logic [DATA_WIDTH-1:0] trig_level,
	input logic [1:0] trig_mode,
	input logic [DATA_WIDTH-1:0] time_scale,
	input logic [1:0] scale_level

);

	// Params ---------------------------------------------------

	// For Screen
	localparam X_LEFT  = 192; // 256 pixels wide (Time Axis)
	localparam X_RIGHT = 448;
	localparam Y_TOP   = 140;
	localparam Y_BOT   = 340; // 200 pixels tall (Voltage Axis)

	// For Scaling
	localparam VIN_MAX = 4024;
	localparam VIN_MIN = 0;
	function logic [9:0] scale (logic [DATA_WIDTH-1:0] data);
	  logic [20:0] temp;
		temp = (data - VIN_MIN)*scale_level;
		temp = temp *(Y_BOT - Y_TOP);
		temp = temp / (VIN_MAX - VIN_MIN);
		scale = Y_BOT - temp;
	endfunction
```

```
// Other
typedef enum {RISING = 0, FALLING = 1, LEVEL = 2, ALWAYS = 3} state_trig ;

// Logic
// Point of no return ---------------------------------------------------
logic [DATA_WIDTH-1:0] sample;
logic [9:0] scaled_volt, scaled_trig;
logic [ADDR_WIDTH-1:0] base_sync;

// sync data
always_ff @(posedge clk) begin
        sample <= data_in;

        if (xpixel == 0 && ypixel == 0) // start of frame
                base_sync <= write_addr;

end

always_comb begin

        // Default values -> For Comb Logic
        outputRGB = 3'b000;
        read_addr   = '0;
        scaled_volt = '0;
        scaled_trig = scale(trig_level);


        // Only compute inside scope region
        if (xpixel >= X_LEFT && xpixel < X_RIGHT) begin
                read_addr = base_sync - (X_RIGHT - xpixel);
                scaled_volt = scale(sample);
        end

        // ---------------- Draw Border
        if ((xpixel == X_LEFT || xpixel == X_RIGHT) &&
                (ypixel >= Y_TOP && ypixel <= Y_BOT))
                outputRGB = 3'b111;

        else if ((ypixel == Y_TOP || ypixel == Y_BOT) &&
                (xpixel >= X_LEFT && xpixel <= X_RIGHT))
                outputRGB = 3'b111;

        // ---------------- Trigger Line
        if ((xpixel >= (X_LEFT - 20) && xpixel < X_LEFT) &&
                (ypixel == scaled_trig || ypixel == scaled_trig - 1)) begin

                case (trig_mode)
                        RISING: outputRGB = 3'b010;  // Rising (Green)
                        FALLING: outputRGB = 3'b100; // Falling (Red)
                        LEVEL: outputRGB = 3'b110;   // Level (Yellow)
                        default: outputRGB = 3'b001; // Free run (Blue)
                endcase
        end

        // ---------------- Waveform
        if ((xpixel > X_LEFT && xpixel < X_RIGHT && ypixel > Y_TOP && ypixel <
```

```
Y_BOT) &&
                        (ypixel == scaled_volt || ypixel == scaled_volt - 1))
                        outputRGB = 3'b011; // waveform line -> Green
        end

endmodule
```

vgacontroller.sv

```
//vgacontroller.sv
// By: Ethan Siemens
// Purpose: Controls the gpio ports based on input, sends current pixel being edited

module vgacontroller
  ( input logic clk, reset,
        input logic [2:0] inputRGB,
        output logic [9:0] xpixel, ypixel,
   output logic [13:0] gpio,
        output  logic vsync, hsync
        ) ;

        // Logic
        logic [15:0] hcount, vcount;
        logic en_vcount;
        logic visible;
        logic[3:0] r, g, b; //VGA Red, Green, Blue

        // Count the pixels
        // Horizontal Count
        horizontal_counter
        hcounter(
                .clk(clk),
                .reset(reset),
                .en_vcounter(en_vcount),
                .hcounter(hcount)
        );

        // Vertical Count
        vertical_counter
        vcounter(
                .clk(clk),
                .reset(reset),
                .en_vcounter(en_vcount),
                .vcounter(vcount)
        );

        // Count Pixels
        assign xpixel = (hcount > 143) ? (hcount - 143) : '0;
  assign ypixel = (vcount > 35) ? (vcount - 35) : '0;

        // Set the sync pulses
        assign hsync = (hcount < 96) ? 1'b1 : 0;
        assign vsync = (vcount < 2) ? 1'b1 : 0;

        // Is the count ready for colour input
        assign visible = (hcount < (784) && hcount > (143)
                                        && vcount < (515) && vcount > (35));
```

```
	// Assign colour
	assign r = visible ? {4{inputRGB[2]}} : '0;
	assign g = visible ? {4{inputRGB[1]}} : '0;
	assign b = visible ? {4{inputRGB[0]}} : '0;

	// Assign to GPIO
	assign gpio[0]  = hsync;
	assign gpio[1]  = vsync;
	assign gpio[5:2]  = r;
	assign gpio[9:6]  = g;
	assign gpio[13:10] = b;

endmodule
```

trigger.sv

```
// trigger.sv
// Tanvir Kaur - Feb 28, 2026

// This module will detect rising, falling, or level trigger events by comparing the current
// ADC sample to the previous sample and the user-selected trigger level.
// This module is controlled by the control.sv

module trigger
#( DATA_WIDTH = 12, ADDR_WIDTH = 8 )
(
	input logic clk, // System Clock
	input logic reset, // Reset
	input logic [DATA_WIDTH-1:0] data_in,   // Incoming Data
	input logic [DATA_WIDTH-1:0] trig_level,// User Decided Trigger Level
	input logic [1:0]  trig_mode, // User Decided Trigger Mode
	output logic trigger_edge // Did the Trigger Fire?
);

	typedef enum {RISING = 0, FALLING = 1, LEVEL = 2, ALWAYS = 3} state_trig ;

	logic [DATA_WIDTH-1:0] prev_data;
	logic condition_met;
	logic trigger_fire;

	always_ff @(posedge clk) begin

		// Reset the previous data
		if (reset)
			prev_data <= '0;

		// No Reset
		else begin

		// Store Data
		prev_data <= data_in;

		// Testing Modes
		case (trig_mode)
			RISING:    condition_met <= (data_in >= trig_level && prev_data <
trig_level);  // Rising Edge
			FALLING:   condition_met <= (data_in <= trig_level && prev_data >
```

```
trig_level);  // Falling Edge
                        LEVEL:    condition_met <= (data_in == trig_level && prev_data ==
trig_level); // Level
                        default:   condition_met <= 1;
                                                                        // Always fire trigger
                endcase

                // Set trigger level
                trigger_fire <= condition_met ? 1 : 0;

                end
        end

        // Get the edge of trigger
        logic trigger_prev;

        always_ff @(posedge clk) begin
                 trigger_prev <= trigger_fire;
        end

        assign trigger_edge = trigger_fire & ~trigger_prev;


endmodule
```

timebase.sv

```
// timebase.sv
// By: Ethan Siemens

module timebase
(   input  logic clk,            // system clock
   input  logic reset,            // reset
   input  logic [11:0] timebase_sel,  // User Input
   output logic sample_tick       // pulse to move capture/read pointer
);

        logic [32:0] counter;
  //logic [32:0] divider;

        // Assign divisions based on User input
        //assign divider = (timebase_sel);

        always_ff @(posedge clk) begin

                // Reset counts
                if(reset) begin
                        counter <= '0;
                        sample_tick <= 0;
                end

                // Count for pulse
                else begin

                        // Reset if count too high -> Put Divider here if need be
                        if(counter > timebase_sel) begin
                                counter <= '0;
                                sample_tick <= 1;
                        end
```

```
                        // Just keep countin
                        else begin
                                counter <= counter + 1'd1;
                                sample_tick <= 0;
                        end
                end
        end

endmodule
```

rotary_encoder.sv

```
//rotary_encoder.sv
//Tanvir Kaur - Feb 28, 2026

module rotary_encoder (
                input logic clk, reset, // Clock
                input logic enca, encb, // Encoder nodes
                output logic up, down // Direction
        );

  logic [3:0] state;
  logic upD, enable;

  always_ff @(posedge clk) begin
    if (reset)
      state <= 4'b0000;
    else
      state <= {state[1:0], enca, encb};
  end

  always_ff @(posedge clk) begin
    if (reset)
      upD <= 1'b0;
    else
      upD <= (state == 4'b1101) ||
          (state == 4'b0100) ||
          (state == 4'b0010) ||
          (state == 4'b1011);
  end

  always_ff @(posedge clk) begin
    if (reset)
      enable <= 1'b0;
    else
      enable <= (state == 4'b1101) ||
            (state == 4'b0100) ||
            (state == 4'b0010) ||
            (state == 4'b1011) ||
            (state == 4'b1110) ||
            (state == 4'b1000) ||
            (state == 4'b0001) ||
            (state == 4'b0111);
  end

  assign up   =  upD & enable;
  assign down = !upD & enable;
```

```
endmodule
```

ltc2308.sv

```
// ltc2308.sv - LTC 2308 ADC interface
// Ed Casas 2026-2-1
// Ethan Siemens
// 2026, Feb 06

// Interface from LTC 2308 500 kHz (2us) 8-channel ADC to Avalon MM
// bus.  Configures the ADC as single-ended unipolar which is how it's
// connected on the DE0-Nano-SoC FPGA board.

// This interface reads sequentially from channels between 'firstch'
// and 'lastch' which are configured by writing a 6-bit value in
// format ({lastch,firstch}). The sample values read from the ADC are
// read as 16-bit values with the channel number in the upper nybble.
// If no data available, the value 16'h8000 is read. Samples are taken
// continuously at 2us per sample.

module ltc2308
  ( input logic clk, reset,    // 50 MHz (12.5ns) clock and reset
    input logic sdo,          // serial data from ADC

    output logic convst, sck, sdi, // ADC control

    output logic [11:0] sample_out,
            output logic adc_ready


    ) ;

          // For cpu
          logic [31:0] readdata; // Avalon MM data bus
  logic [31:0] writedata;
  logic read, write;

  // ADC states and state duration counter

  typedef enum { start, convert, transfer, acquire } state_t ;

  state_t state, state_next ;

  logic [6:0] count, count_next ;

`define set_next(s,c) begin state_next=(s); count_next=$bits(count)'(c)-1'b1; end

  always_comb begin
    if ( reset )
      `set_next ( start, 2 )
    else if ( count )
      `set_next ( state, count )
    else
      case(state)
        start: `set_next ( convert, 126 )
        convert: `set_next ( transfer, 24 )
        transfer: `set_next ( acquire, 8 )
```

```
        acquire: `set_next ( start, 2 )
      endcase
  end

  always_ff @(posedge clk) begin
    state <= state_next ;
    count <= count_next ;
  end

  // ADC control signals
  always_ff @(posedge clk) begin
    convst <= state_next == start ;
    sck <= state_next == transfer && !count_next[0] ;
  end

  // The first/last ADC input multiplexer channels, channel sent to
  // ADC (incremented at end of conversion), configured channel is
  // saved in 'prevch'
  logic [2:0] firstch, lastch, ch, prevch ;

  // Increment the ADC input channel, starting at 'firstch' and
  // wrapping around when it reaches 'lastch'.
//   always_ff @(posedge clk) begin
//     if ( reset ) begin
//       ch <= firstch ;
//       prevch <= firstch ;
//     end else if ( state == convert && !count_next ) begin
//       ch <= ch == lastch ? firstch : ch + 1'b1 ;
//       prevch <= ch ;
//     end
//   end
          assign ch = '0;

  // ADC configuration on SDI
  logic [11:0] adccfg ;
  assign adccfg = { 1'b1, ch[0], ch[2:1], 2'b10, 6'b0 }  ;
  always_ff @(posedge clk)
    sdi <= state_next == transfer ? adccfg[count_next >> 1] : '0 ;

  // ADC data on SDO
  logic [11:0] sample ;
  always_ff @(posedge clk)
    sample <= state == transfer && !sck ? { sample[10:0], sdo } : sample ;

  // A CPU write over the Avalon MM bus sets the first and last
  // channels to be sampled.
  always_ff @(posedge clk)
    if ( write )
      { lastch, firstch } <= 6'b000000;

  // Register the read signal to detect the first clock of a CPU read
  // cycle.
  logic prevread ;
  always_ff @(posedge clk)
    prevread <= read ;

  always_ff @(posedge clk)    begin
```

```
		adc_ready <= (state == transfer && state_next == acquire) ;
		if(adc_ready)
			sample_out <= sample;
	end

endmodule
```

input_control.sv

```
// input_control.sv
// Tanvir Kaur - 02-28-26
// This modules enable the user interface

// idea:
// trig levels are different states that will go from one state to another
// the system will be in rising egde trigger mode by default
// on pressing boosterpack switch the system will go to falling edge mode
// one of the rotary encoder will be used to adjust the trigger level for level triggered mode
// second rotary encoder will be used to adjust the timebase, the number of samples displayed
// second boosterpack switch will reset both the trigger and timebase

module input_control
#( DATA_WIDTH = 12, ADDR_WIDTH = 8 )
(  input logic clk, // 1kHz Clock
	input logic reset, // Reset Button
	input logic trigger, // Trigger Button
  input logic enc1a, enc1b,// Encoder #1 (trigger level)
  input logic enc2a, enc2b, // Encoder #2 (timebase)
	input logic scale,

  // Outputs to oscilloscope
	output logic [DATA_WIDTH-1:0] trig_level, // Trigger Level
  output logic [1:0] trig_mode, // Trigger Mode
  output logic [DATA_WIDTH-1:0] timebase_sel, // Timebase
	output logic [1:0] scale_level // Scale Level
);

	// PARAMETERS TO BE CHANGED
	localparam DEFAULT_LEVEL    = 12'd0;
	localparam DEFAULT_TIMEBASE = 12'd0;
	localparam DEFAULT_MODE     = 2'b0;
	localparam DEFAULT_SCALE    = 2'b1;

	localparam MAX_TRIGGER = 12'd4000;
	localparam MIN_TRIGGER = 12'd0;
	localparam TRIG_SCALE = 12'd10;

	localparam MAX_TIME = 12'd4000;
	localparam MIN_TIME = 12'd1; // -> At Zero Will Freeze Screen
	localparam TIME_SCALE = 12'd1;

	// Encoder Detection
	logic level_up, level_dn;
	logic time_up, time_dn;

	// Trigger Level Encoder
	rotary_encoder
	level_enc (
```

```
		.clk(clk),
		.reset(reset),
		.enca(enc1a),
		.encb(enc1b),
		.down(level_dn),
		.up(level_up)
);

// Time Scaling Encoder
rotary_encoder
time_enc (
		.clk(clk),
		.reset(reset),
		.enca(enc2a),
		.encb(enc2b),
		.down(time_dn),
		.up(time_up)
);


// Button Debounce
logic trigger_p; //Trigger Button Pressed
button
trigger_button (
		.clk(clk),
		.reset(reset),
		.button(trigger),
		.pressed(trigger_p)
);

logic scale_p; //Trigger Button Pressed
button
scale_button (
		.clk(clk),
		.reset(reset),
		.button(scale),
		.pressed(scale_p)
);

// Registers
always_ff @(posedge clk) begin

		// Reset back to defaults
		if (reset) begin
			trig_mode <= DEFAULT_MODE;
			trig_level <= DEFAULT_LEVEL;
			timebase_sel <= DEFAULT_TIMEBASE;
			scale_level <= DEFAULT_SCALE;
		end

		else begin

		// Run through trigger modes
		if (trigger_p)
			trig_mode <= trig_mode + 1'b1;

		if (scale_p)
```

```
                        scale_level <= scale_level == 2'd1 ? 2'd2 :
                                                        scale_level == 2'd2 ? 2'd3 : 2'd1;

                // Adjust trigger level
                if(level_up)
                        trig_level <= (trig_level < MAX_TRIGGER) ? trig_level + TRIG_SCALE :
trig_level;
                else if (level_dn)
                        trig_level <= (trig_level > MIN_TRIGGER) ? trig_level - TRIG_SCALE :
trig_level;

                // Adjust timebase
                if(time_up)
                        timebase_sel <= (timebase_sel < MAX_TIME) ? timebase_sel +
TIME_SCALE : timebase_sel;
                else if (time_dn)
                        timebase_sel <= (timebase_sel > MIN_TIME) ? timebase_sel -
TIME_SCALE : timebase_sel;
                end
        end

endmodule
```

horizontal_counter.sv

```
// horizontal_counter.sv
// By: Ethan Siemens
// Purpose: Counts the horizontal 'pixels' for vga


module horizontal_counter
        #( countMax = 799 )
        ( input logic clk, reset,
   output logic en_vcounter,
         output logic [15:0] hcounter
        );

        always_ff@(posedge clk) begin
                if (reset)begin
                        hcounter <= 0;
                        en_vcounter <= 0;
                end
                else if (hcounter < countMax) begin // counts until 800 - vga standards
                        hcounter <= hcounter + 1;
                        en_vcounter <= 0;
                end

                else begin
                        hcounter <= 0;
                        en_vcounter <= 1;
                end
        end

endmodule
```

vertical_counter.sv

```
// vertical_counter.sv
// By: Ethan Siemens
```

```
// Purpose: Counts the vertical 'pixels' for vga

module vertical_counter
	#( countMax = 524 )
	( input logic clk, reset,
    input logic en_vcounter,
	  output logic [15:0] vcounter
	);

	always_ff@(posedge clk) begin
		if (reset)
			vcounter <= 0;
		else if (en_vcounter == 1) begin
			if (vcounter < countMax) // counts until 525 - vga standards
				vcounter <= vcounter + 1;

			else
				vcounter <= 0;
		end
	end

endmodule
```

clkdiv.sv

```
// clkdiv.sv
// By: Ethan Siemens
// Purpose: Divide Clocks

module clkdiv
  #( fin = 50_000_000, fout = 1000 )
  ( input logic clkin,
    output logic clkout );

  localparam divider = fin / fout / 2 - 1 ;

  logic [$clog2(divider):0] count ;

  always_ff @(posedge clkin)
    count <= !count ? $bits(count)'(divider) : count - 1'b1 ;

  always_ff @(posedge clkin)
    clkout <= count ? clkout : ~clkout ;

endmodule
```

button.sv

```
// button.sv
// By: Ethan Siemens
// debounce button

module button (
	input logic clk, reset, button,
	output logic pressed
	);

	logic button_prev;
```

```
    always_ff @(posedge clk) begin

        if(reset) begin
            button_prev <= 0;
            pressed <= 0;
        end
        else begin
            button_prev <= button;
            if(button && !button_prev) // rising edge of button
                pressed <= 1;
            else
                pressed <= 0;
        end

    end


endmodule
```

References

1. *FPGA-based oscilloscope*. (2007). Cornell University ECE 5760 Final Projects.FPGA Digital Oscilloscope

2. D. Wu, H. Jiao, Y. He and B. Lin, "Innovative Experimental Project Design of a Digital Oscilloscope Using FPGA Technology," 2024 IEEE 2nd International Conference on Electrical, Automation and Computer Engineering (ICEACE), Changchun, China, 2024, pp. 312-316, doi: 10.1109/ICEACE63551.2024.10898918. keywords:
    a. {Costs;Oscilloscopes;Electric variables measurement;User interfaces;Software;Hardware;Liquid crystal displays;Frequency measurement;Hardware design languages;Field programmable gate arrays;FPGA;Digital Oscilloscope;Nios II;LCD;Waveform storage},

3. How to Create VGA Controller in Verilog on FPGA? | Xilinx FPGA Programming Tutorials. (2019). Simple Tutorials for Embedded Systems. https://www.youtube.com/watch?v=4enWoVHCykI